

Classicmodels API paths (main.py)

This notebook exercises the `/customers`, `/orders`, and `/orderdetails` routes from `app/main.py` over HTTP using the `requests` package.

Prerequisites:

1. A running MySQL server with the `classicmodels` sample database loaded.
2. A `.env` file at the repo root (or shell exports) with `MYSQL_HOST`, `MYSQL_PORT`, `MYSQL_USER`, `MYSQL_PASSWORD`, `MYSQL_DATABASE=classicmodels`.
3. The API running from the repo root, e.g. `uvicorn app.main:app --reload --port 8000`.

Default base URL is `http://127.0.0.1:8000`; override with `API_BASE_URL`.

Note: Cells that `POST`, `PUT`, or `DELETE` modify the MySQL `classicmodels` database. The create/update/delete cells use disposable IDs that are cleaned up at the end of each section.

```
In [24]: import os
from pathlib import Path

import requests

# Load MYSQL_* from the repo-root .env so the direct-resource cells below
# can connect even when the API server isn't running.
try:
    from dotenv import load_dotenv

    load_dotenv(Path.cwd().parent / ".env")
except ImportError:
    pass

BASE_URL = os.environ.get("API_BASE_URL", "http://127.0.0.1:8000").rstrip("/")

try:
    _health = requests.get(f"{BASE_URL}/health", timeout=5)
    _health.raise_for_status()
except requests.RequestException as exc:
    raise RuntimeError(
        f"Cannot reach API at {BASE_URL!r}. From the repo root run e.g. "
        "`uvicorn app.main:app --reload --port 8000`, then rerun this cell "
        "(or set API_BASE_URL if the server uses another host/port).")
    from exc

# Sample IDs from the bundled classicmodels database.
SAMPLE_CUSTOMER_NUMBER = 103
```

```
SAMPLE_ORDER_NUMBER = 10100
SAMPLE_PRODUCT_CODE = "S18_1749"
```

Customers

GET /customers

Optional query params act as an equality template (e.g. `country`, `city`, `state`, `salesRepEmployeeNumber`).

```
In [25]: resp = requests.get(f"{BASE_URL}/customers", timeout=30)
         assert resp.status_code == 200, resp.text
         data = resp.json()
         assert "items" in data
         print("count:", len(data["items"]))
         data["items"][:3]
```

count: 122

```
Out[25]: [{'customerNumber': 103,
  'customerName': 'Atelier graphique',
  'contactLastName': 'Schmitt',
  'contactFirstName': 'Carine ',
  'phone': '40.32.2555',
  'addressLine1': '54, rue Royale',
  'addressLine2': None,
  'city': 'Nantes',
  'state': None,
  'postalCode': '44000',
  'country': 'France',
  'salesRepEmployeeNumber': 1370,
  'creditLimit': '21000.00'},
{'customerNumber': 112,
  'customerName': 'Signal Gift Stores',
  'contactLastName': 'King',
  'contactFirstName': 'Jean',
  'phone': '7025551838',
  'addressLine1': '8489 Strong St.',
  'addressLine2': None,
  'city': 'Las Vegas',
  'state': 'NV',
  'postalCode': '83030',
  'country': 'USA',
  'salesRepEmployeeNumber': 1166,
  'creditLimit': '71800.00'},
{'customerNumber': 114,
  'customerName': 'Australian Collectors, Co.',
  'contactLastName': 'Ferguson',
  'contactFirstName': 'Peter',
  'phone': '03 9520 4555',
  'addressLine1': '636 St Kilda Road',
  'addressLine2': 'Level 3',
  'city': 'Melbourne',
  'state': 'Victoria',
  'postalCode': '3004',
  'country': 'Australia',
  'salesRepEmployeeNumber': 1611,
  'creditLimit': '117300.00'}]
```

```
In [26]: resp = requests.get(f"{BASE_URL}/customers", params={"country": "USA"}, timeout=10)
assert resp.status_code == 200
items = resp.json()["items"]
assert all(r["country"] == "USA" for r in items)
len(items)
```

Out[26]: 36

GET /customers/{customerNumber}

Returns **404** if the customer does not exist.

```
In [27]: resp = requests.get(f"{BASE_URL}/customers/{SAMPLE_CUSTOMER_NUMBER}", timeout=10)
assert resp.status_code == 200
```

```
resp.json()
```

```
Out[27]: {'customerNumber': 103,  
         'customerName': 'Atelier graphique',  
         'contactLastName': 'Schmitt',  
         'contactFirstName': 'Carine ',  
         'phone': '40.32.2555',  
         'addressLine1': '54, rue Royale',  
         'addressLine2': None,  
         'city': 'Nantes',  
         'state': None,  
         'postalCode': '44000',  
         'country': 'France',  
         'salesRepEmployeeNumber': 1370,  
         'creditLimit': '21000.00'}
```

```
In [28]: missing = requests.get(f"{BASE_URL}/customers/999999", timeout=30)  
assert missing.status_code == 404  
missing.json()
```

```
Out[28]: {'detail': "No customer with customerNumber '999999'"}
```

POST /customers

`customerNumber` is required (classicmodels does not auto-increment). The cell uses a high disposable number; the **DELETE** cell below cleans it up.

```
In [29]: NEW_CUSTOMER_NUMBER = 999001  
payload = {  
    "customerNumber": NEW_CUSTOMER_NUMBER,  
    "customerName": "Notebook Test Co",  
    "contactLastName": "Tester",  
    "contactFirstName": "Note",  
    "phone": "555-0100",  
    "addressLine1": "1 Notebook Way",  
    "city": "NYC",  
    "country": "USA",  
    "creditLimit": 1000.00,  
}  
resp = requests.post(f"{BASE_URL}/customers", json=payload, timeout=30)  
assert resp.status_code == 200, resp.text  
resp.json()
```

```
Out[29]: '999001'
```

PUT /customers/{customerNumber}

Updates by id; returns **404** when the row does not exist.

```
In [30]: update_body = {  
    "customerName": "Notebook Test Co (Updated)",  
    "contactLastName": "Tester",  
}
```

```

        "contactFirstName": "Note",
        "phone": "555-0101",
        "addressLine1": "2 Notebook Way",
        "city": "NYC",
        "country": "USA",
        "creditLimit": 2500.00,
    }
    resp = requests.put(f"{BASE_URL}/customers/{NEW_CUSTOMER_NUMBER}", json=update_data)
    assert resp.status_code == 200, resp.text
    assert resp.json() == {"updated": 1}
    requests.get(f"{BASE_URL}/customers/{NEW_CUSTOMER_NUMBER}", timeout=30).json()

```

```

Out[30]: {'customerNumber': 999001,
          'customerName': 'Notebook Test Co (Updated)',
          'contactLastName': 'Tester',
          'contactFirstName': 'Note',
          'phone': '555-0101',
          'addressLine1': '2 Notebook Way',
          'addressLine2': None,
          'city': 'NYC',
          'state': None,
          'postalCode': None,
          'country': 'USA',
          'salesRepEmployeeNumber': None,
          'creditLimit': '2500.00'}

```

DELETE /customers/{customerNumber}

```

In [31]: resp = requests.delete(f"{BASE_URL}/customers/{NEW_CUSTOMER_NUMBER}", timeout=30)
         assert resp.status_code == 200
         assert resp.json() == {"deleted": 1}

         gone = requests.get(f"{BASE_URL}/customers/{NEW_CUSTOMER_NUMBER}", timeout=30)
         assert gone.status_code == 404
         gone.json()

```

```

Out[31]: {'detail': "No customer with customerNumber '999001'"}

```

Orders

GET /orders and GET /orders/{orderNumber}

```

In [32]: resp = requests.get(f"{BASE_URL}/orders", timeout=30)
         assert resp.status_code == 200
         items = resp.json()["items"]
         print("count:", len(items))
         items[:3]

```

```
count: 326
```

```
Out[32]: [{'orderNumber': 10100,
          'orderDate': '2003-01-06',
          'requiredDate': '2003-01-13',
          'shippedDate': '2003-01-10',
          'status': 'Shipped',
          'comments': None,
          'customerNumber': 363},
         {'orderNumber': 10101,
          'orderDate': '2003-01-09',
          'requiredDate': '2003-01-18',
          'shippedDate': '2003-01-11',
          'status': 'Shipped',
          'comments': 'Check on availability.',
          'customerNumber': 128},
         {'orderNumber': 10102,
          'orderDate': '2003-01-10',
          'requiredDate': '2003-01-18',
          'shippedDate': '2003-01-14',
          'status': 'Shipped',
          'comments': None,
          'customerNumber': 181}]
```

```
In [33]: resp = requests.get(
          f"{BASE_URL}/orders", params={"customerNumber": SAMPLE_CUSTOMER_NUMBER},
        )
assert resp.status_code == 200
items = resp.json()["items"]
assert all(r["customerNumber"] == SAMPLE_CUSTOMER_NUMBER for r in items)
len(items)
```

```
Out[33]: 3
```

```
In [34]: resp = requests.get(f"{BASE_URL}/orders/{SAMPLE_ORDER_NUMBER}", timeout=30)
assert resp.status_code == 200
resp.json()
```

```
Out[34]: {'orderNumber': 10100,
          'orderDate': '2003-01-06',
          'requiredDate': '2003-01-13',
          'shippedDate': '2003-01-10',
          'status': 'Shipped',
          'comments': None,
          'customerNumber': 363}
```

```
In [35]: missing = requests.get(f"{BASE_URL}/orders/99999999", timeout=30)
assert missing.status_code == 404
missing.json()
```

```
Out[35]: {'detail': "No order with orderNumber '99999999'"} 
```

POST /orders + PUT + DELETE

Creates an order under an existing `customerNumber`, updates it, then deletes it. The order is also used by the order-details section below; the cleanup cell at the bottom of

the notebook removes it.

```
In [36]: NEW_ORDER_NUMBER = 999001
payload = {
    "orderNumber": NEW_ORDER_NUMBER,
    "orderDate": "2026-05-01",
    "requiredDate": "2026-05-15",
    "status": "In Process",
    "customerNumber": SAMPLE_CUSTOMER_NUMBER,
}
resp = requests.post(f"{BASE_URL}/orders", json=payload, timeout=30)
assert resp.status_code == 200, resp.text
resp.json()
```

Out[36]: '999001'

```
In [37]: update_body = {
    "orderDate": "2026-05-01",
    "requiredDate": "2026-05-20",
    "status": "Shipped",
    "shippedDate": "2026-05-05",
    "customerNumber": SAMPLE_CUSTOMER_NUMBER,
}
resp = requests.put(f"{BASE_URL}/orders/{NEW_ORDER_NUMBER}", json=update_body)
assert resp.status_code == 200, resp.text
assert resp.json() == {"updated": 1}
requests.get(f"{BASE_URL}/orders/{NEW_ORDER_NUMBER}", timeout=30).json()
```

Out[37]: {'orderNumber': 999001,
'orderDate': '2026-05-01',
'requiredDate': '2026-05-20',
'shippedDate': '2026-05-05',
'status': 'Shipped',
'comments': None,
'customerNumber': 103}

Order details (composite PK: orderNumber + productCode)

GET /orderdetails (collection)

And GET /orders/{orderNumber}/orderdetails (lines for one order).

```
In [38]: resp = requests.get(f"{BASE_URL}/orderdetails", timeout=30)
assert resp.status_code == 200
print("count:", len(resp.json()["items"]))
```

count: 2996

```
In [39]: resp = requests.get(
    f"{BASE_URL}/orders/{SAMPLE_ORDER_NUMBER}/orderdetails", timeout=30
)
```

```

assert resp.status_code == 200
items = resp.json()["items"]
assert all(r["orderNumber"] == SAMPLE_ORDER_NUMBER for r in items)
items

```

```

Out[39]: [{ 'orderNumber': 10100,
            'productCode': 'S18_1749',
            'quantityOrdered': 30,
            'priceEach': '136.00',
            'orderLineNumber': 3},
          { 'orderNumber': 10100,
            'productCode': 'S18_2248',
            'quantityOrdered': 50,
            'priceEach': '55.09',
            'orderLineNumber': 2},
          { 'orderNumber': 10100,
            'productCode': 'S18_4409',
            'quantityOrdered': 22,
            'priceEach': '75.46',
            'orderLineNumber': 4},
          { 'orderNumber': 10100,
            'productCode': 'S24_3969',
            'quantityOrdered': 49,
            'priceEach': '35.29',
            'orderLineNumber': 1}]

```

```

In [40]: resp = requests.get(
            f"{BASE_URL}/orders/{SAMPLE_ORDER_NUMBER}/orderdetails/{SAMPLE_PRODUCT_CODE}",
            timeout=30,
        )
assert resp.status_code == 200, resp.text
resp.json()

```

```

Out[40]: { 'orderNumber': 10100,
            'productCode': 'S18_1749',
            'quantityOrdered': 30,
            'priceEach': '136.00',
            'orderLineNumber': 3}

```

POST /orderdetails, PUT and DELETE on /orders/{orderNumber}/orderdetails/{productCode}

Adds, updates, and removes a line on the test order created above.

```

In [41]: NEW_PRODUCT_CODE = SAMPLE_PRODUCT_CODE
payload = {
    "orderNumber": NEW_ORDER_NUMBER,
    "productCode": NEW_PRODUCT_CODE,
    "quantityOrdered": 5,
    "priceEach": 99.99,
    "orderLineNumber": 1,
}
resp = requests.post(f"{BASE_URL}/orderdetails", json=payload, timeout=30)

```

```
assert resp.status_code == 200, resp.text
resp.json()
```

Out[41]: '999001,S18_1749'

```
In [42]: update_body = {
    "quantityOrdered": 10,
    "priceEach": 89.99,
    "orderLineNumber": 1,
  }
resp = requests.put(
    f"{BASE_URL}/orders/{NEW_ORDER_NUMBER}/orderdetails/{NEW_PRODUCT_CODE}",
    json=update_body,
    timeout=30,
)
assert resp.status_code == 200, resp.text
assert resp.json() == {"updated": 1}
requests.get(
    f"{BASE_URL}/orders/{NEW_ORDER_NUMBER}/orderdetails/{NEW_PRODUCT_CODE}",
).json()
```

Out[42]: {'orderNumber': 999001,
'productCode': 'S18_1749',
'quantityOrdered': 10,
'priceEach': '89.99',
'orderLineNumber': 1}

```
In [43]: resp = requests.delete(
    f"{BASE_URL}/orders/{NEW_ORDER_NUMBER}/orderdetails/{NEW_PRODUCT_CODE}",
)
assert resp.status_code == 200
assert resp.json() == {"deleted": 1}
```

Cleanup: remove the test order created above

```
In [44]: resp = requests.delete(f"{BASE_URL}/orders/{NEW_ORDER_NUMBER}", timeout=30)
assert resp.status_code == 200
resp.json()
```

Out[44]: {'deleted': 1}

Calling the resource layer directly

Skips HTTP and exercises `MySQLDataService` through the resource classes. Requires the same MySQL config.

```
In [45]: import sys
from pathlib import Path

sys.path.insert(0, str(Path.cwd().parent))

from app.resources.CustomerResource import CustomerResource
```

```
from app.resources.OrderResource import OrderResource
from app.resources.OrderDetailsResource import OrderDetailsResource

cust = CustomerResource()
one = cust.get_by_id(str(SAMPLE_CUSTOMER_NUMBER))
one
```

Out [45]: Customer(customerNumber=103, customerName='Atelier graphique', contactLastName='Schmitt', contactFirstName='Carine ', phone='40.32.2555', addressLine1='54, rue Royale', addressLine2=None, city='Nantes', state=None, postalCode='44000', country='France', salesRepEmployeeNumber=1370, creditLimit=Decimal('21000.00'))

```
In [46]: ords = OrderResource()
ords.get({"customerNumber": SAMPLE_CUSTOMER_NUMBER}).items[:3]
```

Out [46]: [Order(orderNumber=10123, orderDate=datetime.date(2003, 5, 20), requiredDate=datetime.date(2003, 5, 29), shippedDate=datetime.date(2003, 5, 22), status='Shipped', comments=None, customerNumber=103),
Order(orderNumber=10298, orderDate=datetime.date(2004, 9, 27), requiredDate=datetime.date(2004, 10, 5), shippedDate=datetime.date(2004, 10, 1), status='Shipped', comments=None, customerNumber=103),
Order(orderNumber=10345, orderDate=datetime.date(2004, 11, 25), requiredDate=datetime.date(2004, 12, 1), shippedDate=datetime.date(2004, 11, 26), status='Shipped', comments=None, customerNumber=103)]

```
In [47]: od = OrderDetailsResource()
od.get_by_order(SAMPLE_ORDER_NUMBER).items
```

Out [47]: [OrderDetail(orderNumber=10100, productCode='S18_1749', quantityOrdered=30, priceEach=Decimal('136.00'), orderLineNumber=3),
OrderDetail(orderNumber=10100, productCode='S18_2248', quantityOrdered=50, priceEach=Decimal('55.09'), orderLineNumber=2),
OrderDetail(orderNumber=10100, productCode='S18_4409', quantityOrdered=22, priceEach=Decimal('75.46'), orderLineNumber=4),
OrderDetail(orderNumber=10100, productCode='S24_3969', quantityOrdered=49, priceEach=Decimal('35.29'), orderLineNumber=1)]